

Part I — GRPO Finetuning of gemma-3-1b-it on GSM8K

Multi-Agent Systems and Agentic AI — Practical Examination

Team **chmawa** (sm3035, bc654, zw499) · written individually by Zixuan Wang (zw499)

Jointly owned (shared team work): the v6e-1 TPU, the seven shared GRPO runs (bdbugenj and R0-R5), and the code repository. All prose, figure selection and interpretation below are my own.

Code: github.com/wdk0082/a8-grpo (to be ported to GitLab for submission) | W&B: [a8-grpo](#) | [GRPO](#), [DAPO](#), [LoRA](#), [GSM8K](#)

I.1 Reproducing the baseline

(i) **The run.** Shared baseline run ([bdbugenj](#)): upstream commit [borisbolliet/tpu-2026077c5a67](#), unmodified `config.py`, full **3364** GRPO steps (one epoch), **4 h 43 m** wall-clock on **v6e-1**.

(ii) **Accuracy.** GSM8K *test* split (materialised from TFDS, shuffle seed 42, $n=64$, greedy, exact numeric match). Base **gemma-3-1b-it**: **51.6%**; LoRA checkpoint **28.1%** at its best *surviving* step (2000; [bdbugenj](#)'s earlier, better checkpoints were pruned) and **3.1%** at step 3364. The baseline *over-optimises*: exact-match correctness falls $52 \rightarrow 28 \rightarrow 6 \rightarrow 3\%$ across steps 0/2000/3000/3364 while the format-shaping reward saturates (it reward-hacks the template). At $n=64$ the CI is ± 12 pt, so we firm the base figure to **47.4%** on the full 1319-example test in I.3.

(iii) **Training curves.** Fig. 1 (grey = baseline): mean reward $\bar{r}$ peaks $\approx$ step 450 then declines, and $\text{KL}(\pi_\theta || \pi_{\text{ref}})$ rises $0.16 \rightarrow \sim 0.5$ with late spikes — the textbook over-optimisation arc.

Baseline patches (as-shipped fixes; full diff: `tpu-2026_our_changes.patch` in repo). (1) `run_tmux.sh` hard-coded a foreign home path; (2) the W&B entity defaulted to a third party — redirected to **a8-grpo**; (3) `evaluate.py` built a fresh $B=0$ LoRA and never restored a checkpoint, so it could only score *base* — we added `Orbax -step/-ckpt-dir` LoRA restore; (4) TFDS's `gsm8k` builder crashes under `protobuf 7.x` once JAX imports, so we pre-materialise the seed-42 test split (`prepare_test_tfds.py`) and added an HF `datasets` fallback.

I.2 Understanding the baseline

(a) **Policies & trainable parameters.** π_θ is the LoRA-adapted policy: frozen **gemma-3-1b-it** weights plus low-rank adapters $\Delta W = BA$; **only A, B are trainable** (the base is frozen). π_{old} is the policy that generated the current rollouts — with `NUM_ITERATIONS  $\mu=1$` there is one gradient step per rollout batch, so $\pi_{\text{old}} = \pi_\theta$ at the update. π_{ref} is the frozen initial policy for the KL penalty; LoRA initialises $B=0$, so $\pi_\theta = \pi_{\text{ref}}$ exactly at step 0.

(b) **Group size & estimator.** $K = \text{NUM_GENERATIONS} = 2$. Tunix uses `advantage_estimator='grpo'`: the **standard** group-mean, std-normalised advantage $\hat{A}_i = (r_i - \bar{r}) / \sigma_r$ with $\bar{r}, \sigma_r$ over all K samples *including i* — *not* leave-one-out.

(c) **Shaping vs. correctness rewards.** Shaping: `match_format_exactly` and `match_format_approximately` (reward the `<reasoning>/<answer>` template). True correctness: `check_answer` and `check_numbers` (extracted number vs. gold). With the correctness reward *alone*, an early policy is almost never numerically right, so every sample in a group scores ≈ 0 : within-group $\sigma_r \rightarrow 0$ and $\hat{A}_i = (r_i - \bar{r}) / \sigma_r$ has no signal (an ε in the denominator averts a literal $0/0$) — degenerate groups, no gradient. The shaping terms give differently-formatted samples different rewards, restoring $\sigma_r > 0$ and a usable advantage that bootstraps learning (the same $\sigma_r \rightarrow 0$ degeneracy analysed in I.4 Q1).

(d) **Clipped surrogate & ϵ .** The PPO-style clip lives in Tunix's `GRPOLearner` (module `tunix.rl`), in its clipped policy-gradient loss: it forms a **per-token** ratio $\rho_t = \pi_\theta(a_t) / \pi_{\text{old}}(a_t)$ and optimises $\min(\rho_t \hat{A}, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A})$, with $\epsilon = \text{EPSILON} = 0.2$. Two deviations from the lecture form: (i) the ratio is **per-token**, not per-sequence; (ii) at $\mu=1$, $\pi_{\text{old}} = \pi_\theta \Rightarrow \rho_t \equiv 1$, so the clip is **dormant** in the baseline (it engages only for $\mu > 1$).

Table 1: Held-out accuracy on the *full* GSM8K test ($n=1319$; HF and TFDS differ only in seed-42 ordering, so at full n the I.1→I.3 source switch is immaterial), greedy/exact-match, $\pm 95\%$ bootstrap CI. Last column: *paired* bootstrap Δ vs. base over the shared prompts (`paired_ci.py`). * = paired 95% CI excludes 0; † = collapse. R1($G2, \beta.3$) and R2($G2, +\text{len}$) fell *below* base (R1 best 37.2 / final 0.0; R2 best 32.8 / final 40.6) — clean negative results.

Model	(G, β) , step	Acc. (%)	95% CI	Δ base (paired)
Base <code>gemma-3-1b-it</code>	—	47.4	44.7–50.0	—
R0 (baseline re-run)	(2, .08) best 700	46.7	44.0–49.4	−0.7 n.s.
R0	(2, .08) final 3364	6.2	4.9–7.5	−41.2†
R4	(2, 0) best 1300	52.8	50.1–55.5	+5.4*
R4	(2, 0) final 3364	48.6	45.9–51.3	+1.2 n.s.
R3b	(8, 0) best 1000	55.7	53.0–58.2	+8.3*
R3b	(8, 0) final 3364	57.2	54.6–59.9	+9.9*
R5	(8, .08) best 1300	53.4	50.7–56.2	+6.1*
R5	(8, .08) final 3364	55.6	52.8–58.2	+8.2*

I.3 Improving on the baseline

Modification & motivation. The baseline over-optimises at $K=2$. The theory of I.4 Q1 gives the GRPO gradient variance $\text{Var}(\hat{g}) = K^{-2} \sum_i (h_i - \bar{h})(h_i - \bar{h})^\top = O(1/K)$, with effective sample size $K_{\text{eff}} \propto K$: a larger group directly lowers the variance of the gradient estimate $\hat{g}$. We therefore take **group size** G (the K of I.2(b) and the theory) as the primary lever and run a controlled $\beta \times G$ **2×2** (KL-penalty weight $\times$ group size) to separate it from the KL leash: R4($G2, \beta0$), R0($G2, \beta.08$; baseline re-run), R3b($G8, \beta0$), R5($G8, \beta.08$) — plus single-axis controls R1($\beta=0.3$) and R2(+soft length penalty), for **six full runs**. All share data (HF GSM8K, seed 42), eval, horizon (3364 steps) and a seed-matched rollout RNG (`PRNGKey(0)`); only the named knob changes per run. R0 re-runs the baseline config with *dense* checkpoints, so its best-val 46.7% recovers the peak that `bdbugenj`’s pruning hid behind I.1’s 28.1%.

We report **(a)** held-out accuracy (Table 1), **(b)** mean reward and KL for all runs on shared axes (Fig. 1), and **(c)** a diagnostic for a known GRPO failure mode — group degeneracy at $G=8$ (Fig. 2b).

(d) Findings vs. theory. $G=8$ is the winning intervention. Both $G=8$ runs beat base — R3b +9.9, R5 +8.2pp, paired CIs excluding 0 — and each beats its matched $G=2$ run (R3b–R4 = +8.6pp at final; R5–R0 = +49pp). **R3b ($G=8, \beta=0$) is the best model at 57.2%.** The 2×2 (Fig. 2a) localises the failure: *the collapse is confined to the $G=2, \beta>0$ corner* (R0 final 6.2%, R1 0%), while the *same $\beta=0.08$ at $G=8$ (R5) is stable*, and at $G=8$ the KL weight is statistically irrelevant (R3b vs. R5: +1.7pp, n.s.). Furthermore *over-optimisation is itself a $G=2$ phenomenon*: $G=2$ peaks at best-val then degrades (R4 52.8→48.6; R0 collapses), whereas $G=8$ **improves monotonically to the final step** (final > best). This is precisely the I.4 Q1 mechanism: larger K shrinks $\text{Var}(\hat{g})$, so updates are smoother — the policy neither reward-hacks into R0’s format-only collapse nor (once $\beta>0$) trips the $\exp(\log\text{-ratio})$ KL-estimator instability behind R1’s blow-up.

Contradicted prediction. We predicted an over-optimising baseline would need a *tighter* KL leash. The opposite held: raising β to 0.3 (R1) collapsed *faster* (0%), and removing the KL term entirely (R4, $\beta=0$) was fine — the cure was variance reduction (group size), not regularisation, echoing DAPO’s removal of the KL term for long-output RL. The diagnostic (Fig. 2b) shows the cost of $G=8$: $\approx \frac{1}{3}$ of groups are degenerate (all- K -equal $\Rightarrow$ zero advantage), the K_{eff} /dead-group issue of I.4 Q1 and the target of DAPO’s Dynamic Sampling — yet the net effect of more rollouts is still +8–10pp. R1 and R2 fell below base (Table 1), honest negative results that further support variance, not extra shaping or a stronger leash, as the operative lever.

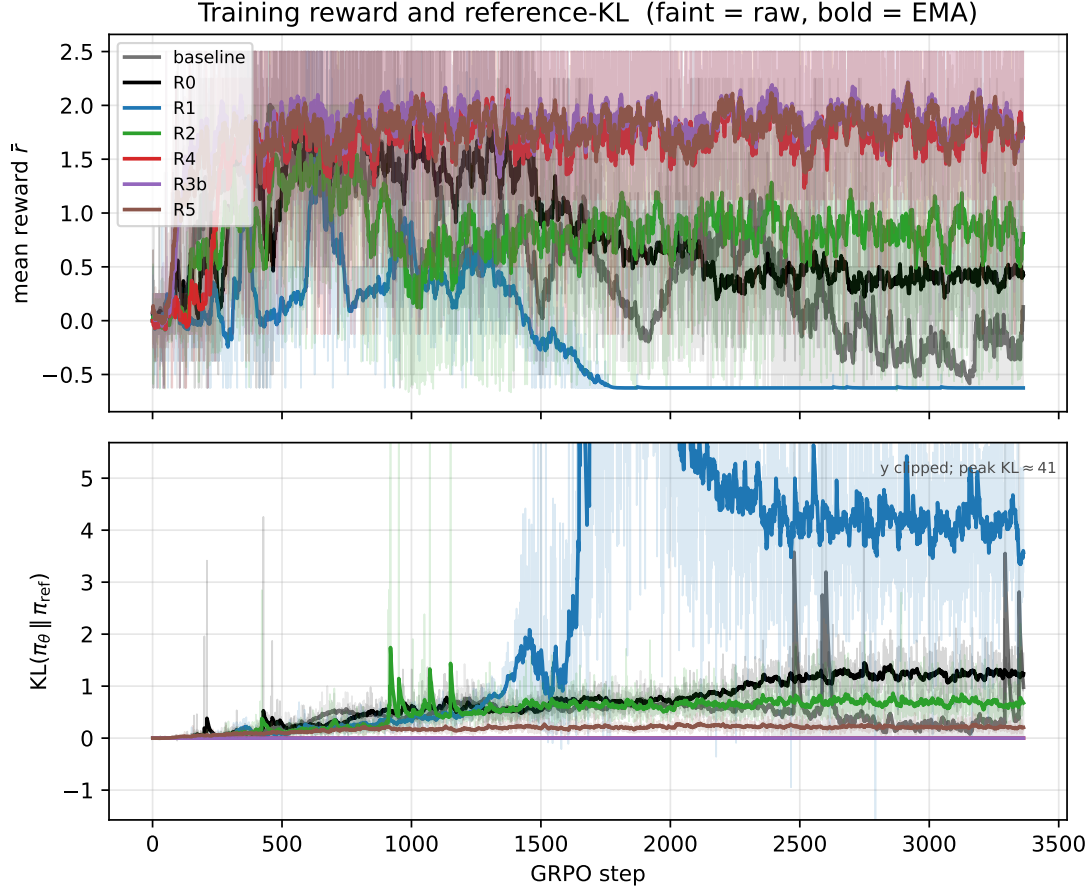


Figure 1: Mean training reward $\bar{r}$ (top) and $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$ (bottom) vs. GRPO step, baseline and all variants on shared axes. Baseline/R0/R1 over-optimize (reward peaks then collapses; KL spikes); the $G=8$ runs (R3b, R5) and R4 stay stable.

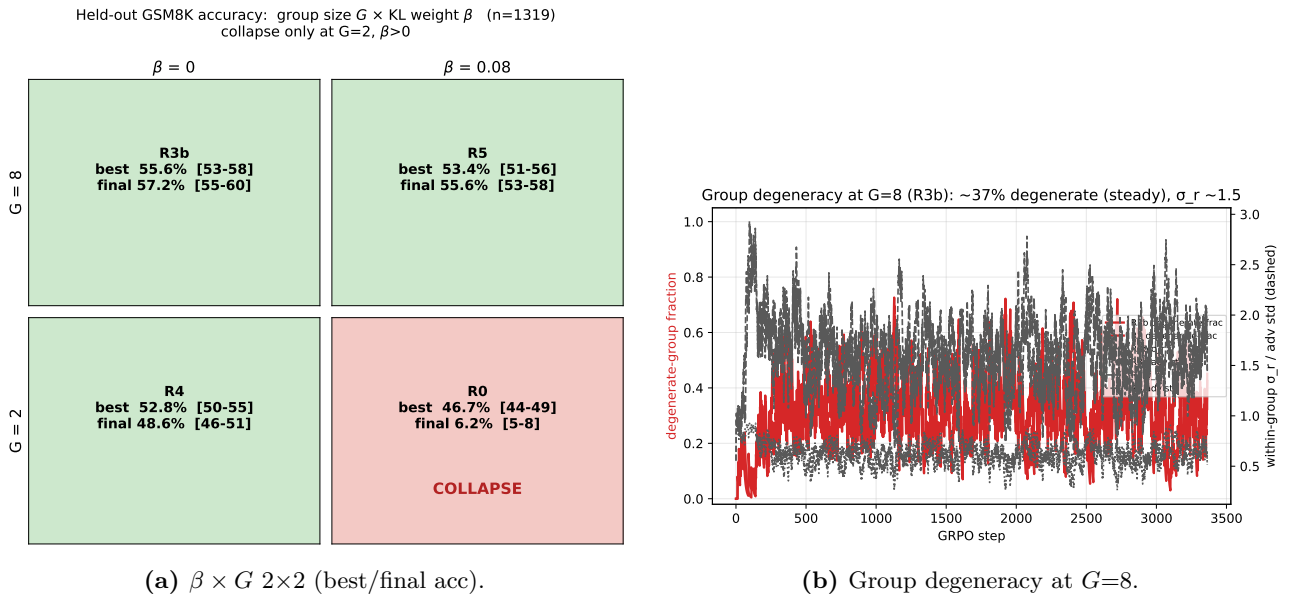


Figure 2: Left: accuracy by (G, β) — collapse occurs *only* at $G=2, \beta>0$ (R0). Right: the diagnostic — at $G=8$ about a third of groups are degenerate (all- K rewards equal) yet σ_r holds ≈ 1.5 (steady, not $\rightarrow 0$).

I.4 GRPO theory — written questions (40 marks)

1. Group-mean normalisation. Throughout this question the actions $a_1, \dots, a_K$ and hence the score vectors $h_i = \nabla_{\theta} \log \pi_{\theta}(a_i | q)$ are fixed. Only the rewards are random.

(i) Since $\mathbb{E}[r_i] = \mu$ and $\mathbb{E}[\bar{r}] = \mu$,

$$\mathbb{E}[\hat{A}_i] = \mathbb{E}\left[\frac{r_i - \bar{r}}{\sigma}\right] = 0.$$

Therefore

$$\mathbb{E}[X_i] = h_i \mathbb{E}[\hat{A}_i] = 0, \quad \mathbb{E}[\hat{g}] = \frac{1}{K} \sum_{i=1}^K \mathbb{E}[X_i] = 0.$$

This does not mean that the true policy gradient vanishes: here we have conditioned on fixed responses a_i , and the reward randomness is independent of the policy. In the real policy-gradient problem the actions themselves are sampled from the policy, and rewards depend on those actions.

(ii) The variables X_i are dependent because all advantages contain the common random term

$$\bar{r} = \frac{1}{K} \sum_{\ell=1}^K r_{\ell}.$$

For $i \neq j$,

$$\text{Cov}(X_i, X_j) = h_i h_j^{\top} \text{Cov}(\hat{A}_i, \hat{A}_j) \tag{1}$$

$$= \frac{h_i h_j^{\top}}{\sigma^2} \mathbb{E}[(r_i - \bar{r})(r_j - \bar{r})]. \tag{2}$$

Now

$$\mathbb{E}[r_i r_j] = \mu^2, \quad \mathbb{E}[r_i \bar{r}] = \mu^2 + \frac{\sigma^2}{K}, \quad \mathbb{E}[\bar{r}^2] = \mu^2 + \frac{\sigma^2}{K}.$$

Hence

$$\mathbb{E}[(r_i - \bar{r})(r_j - \bar{r})] = \mu^2 - 2\left(\mu^2 + \frac{\sigma^2}{K}\right) + \left(\mu^2 + \frac{\sigma^2}{K}\right) \tag{3}$$

$$= -\frac{\sigma^2}{K}. \tag{4}$$

Therefore

$$\boxed{\text{Cov}(X_i, X_j) = -\frac{1}{K} h_i h_j^{\top}, \quad i \neq j.}$$

The scalar coefficient is negative. Intuitively, after subtracting the group mean, if one reward is above the group mean then the others are, on average, pushed below it, since

$$\sum_{i=1}^K (r_i - \bar{r}) = 0.$$

(iii) First,

$$\text{Var}(\hat{A}_i) = \frac{1}{\sigma^2} \text{Var}(r_i - \bar{r}) = 1 - \frac{1}{K},$$

so

$$\text{Var}(X_i) = \left(1 - \frac{1}{K}\right) h_i h_i^{\top}.$$

Thus

$$\text{Var}(\hat{g}) = \frac{1}{K^2} \left[\sum_{i=1}^K \text{Var}(X_i) + \sum_{i \neq j} \text{Cov}(X_i, X_j) \right] \quad (5)$$

$$= \frac{1}{K^2} \left[\left(1 - \frac{1}{K}\right) \sum_{i=1}^K h_i h_i^\top - \frac{1}{K} \sum_{i \neq j} h_i h_j^\top \right]. \quad (6)$$

Let

$$H = \sum_{i=1}^K h_i, \quad \bar{h} = \frac{1}{K} H.$$

Since

$$\sum_{i \neq j} h_i h_j^\top = H H^\top - \sum_{i=1}^K h_i h_i^\top,$$

we obtain the compact form

$$\text{Var}(\hat{g}) = \frac{1}{K^2} \left[\sum_{i=1}^K h_i h_i^\top - \frac{1}{K} H H^\top \right] = \frac{1}{K^2} \sum_{i=1}^K (h_i - \bar{h})(h_i - \bar{h})^\top.$$

The naive independent-sample calculation would give

$$\text{Var}_{\text{naive}}(\hat{g}) = \frac{1}{K} \text{Var}(X_1) = \frac{1}{K} \left(1 - \frac{1}{K}\right) h_1 h_1^\top,$$

which misses the negative cross-covariances induced by the shared baseline $\bar{r}$.

Because the variance is matrix-valued, a scalar effective sample size requires a convention. Along any direction $u \neq 0$, define $K_{\text{eff}}(u)$ by

$$u^\top \text{Var}(\hat{g}) u = \frac{1}{K_{\text{eff}}(u)} u^\top \text{Var}(X_1) u.$$

Then

$$K_{\text{eff}}(u) = \frac{K^2 \left(1 - \frac{1}{K}\right) (u^\top h_1)^2}{\sum_{i=1}^K (u^\top h_i)^2 - \frac{1}{K} \left(\sum_{i=1}^K u^\top h_i\right)^2}.$$

Equivalently, a trace-based scalar summary is

$$K_{\text{eff}}^{\text{tr}} = \frac{\text{tr} \text{Var}(X_1)}{\text{tr} \text{Var}(\hat{g})} = \frac{K^2 \left(1 - \frac{1}{K}\right) \|h_1\|^2}{\sum_{i=1}^K \|h_i\|^2 - \frac{1}{K} \left\| \sum_{i=1}^K h_i \right\|^2}.$$

For large K , if the empirical variance of the score vectors is $O(1)$, then $\text{Var}(\hat{g}) = O(1/K)$, so the effective sample size grows linearly in K , up to constants depending on the dispersion of the h_i . If all h_i are aligned, say $h_i = c_i v$, then

$$\text{Var}(\hat{g}) = \frac{v v^\top}{K^2} \sum_{i=1}^K (c_i - \bar{c})^2.$$

Thus only variation in the magnitudes c_i contributes. If all h_i are identical, then $\text{Var}(\hat{g}) = 0$, because

$$\hat{g} = \frac{h}{K} \sum_{i=1}^K \hat{A}_i = 0.$$

2. KL-regularised policy update.

(i) The objective is

$$F(\pi) = \sum_a \pi(a) A_t(a) - \frac{1}{\eta} \sum_a \pi(a) \log \frac{\pi(a)}{\pi_t(a)}.$$

Using a Lagrange multiplier λ for $\sum_a \pi(a) = 1$, the stationarity condition is

$$A_t(a) - \frac{1}{\eta} \left(\log \frac{\pi(a)}{\pi_t(a)} + 1 \right) + \lambda = 0.$$

Hence

$$\log \frac{\pi(a)}{\pi_t(a)} = \eta A_t(a) + \eta \lambda - 1,$$

so the optimiser has the form

$$\pi_{t+1}(a) = \frac{\pi_t(a) \exp(\eta A_t(a))}{Z_t}, \quad Z_t = \sum_b \pi_t(b) \exp(\eta A_t(b)).$$

Thus

$$\boxed{\pi_{t+1}(a) = \frac{\pi_t(a) \exp(\eta A_t(a))}{\mathbb{E}_{b \sim \pi_t} [\exp(\eta A_t(b))]}.$$

The optimiser is unique on the support of π_t , since the objective is strictly concave in π . The parameter η is a step-size: as $\eta \rightarrow 0$, $\pi_{t+1} \rightarrow \pi_t$; as η grows, the update increasingly concentrates mass on actions with large advantage.

(ii) Let

$$J(\pi) = \mathbb{E}_{a \sim \pi} [A^{\pi_t}(a)].$$

For the exact advantage,

$$J(\pi_t) = \mathbb{E}_{a \sim \pi_t} [A^{\pi_t}(a)] = 0.$$

The objective value at $\pi = \pi_t$ is also zero, because

$$J(\pi_t) - \frac{1}{\eta} \text{KL}(\pi_t \| \pi_t) = 0.$$

Since π_{t+1} maximises the regularised objective,

$$J(\pi_{t+1}) - \frac{1}{\eta} \text{KL}(\pi_{t+1} \| \pi_t) \geq 0.$$

Therefore

$$J(\pi_{t+1}) \geq \frac{1}{\eta} \text{KL}(\pi_{t+1} \| \pi_t) \geq 0 = J(\pi_t).$$

Hence the update gives monotone improvement in the scalar surrogate J .

In neural-network GRPO this guarantee can fail because the exact optimisation over all distributions is replaced by stochastic gradient updates in a restricted parametric family, using finite-sample, noisy advantage estimates. It can also fail because the PPO-style clipped surrogate is not the same as the exact KL-regularised objective.

(iii) The clipped objective uses

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | q)}{\pi_{\text{old}}(a_t | q)}.$$

For $\hat{A}_t > 0$, increasing ρ_t improves the objective only until $\rho_t = 1 + \epsilon$; beyond that, the clipped term is flat. For $\hat{A}_t < 0$, decreasing ρ_t improves the objective only until $\rho_t = 1 - \epsilon$. Thus clipping induces a soft, samplewise trust region

$$\rho_t(\theta) \in [1 - \epsilon, 1 + \epsilon]$$

in the advantage-improving direction.

This differs from a joint KL trust region

$$\text{KL}(\pi_\theta(\cdot | q) \| \pi_{\text{old}}(\cdot | q)) \leq \delta.$$

The KL constraint is distribution-level and couples all actions through one global constraint. The PPO clip is local to sampled actions, is not a hard constraint on the full policy, and does not directly control the probability of unsampled actions. Consequently, the clipped objective can still allow large KL drift even if many sampled ratios are clipped.

3. Mixture proposal and importance weighting. Write

$$p(a) = \pi_{\text{old}}(a | q), \quad u(a) = \pi_{\text{unif}}(a | q), \quad m(a) = \pi_{\text{mix}}(a | q) = (1 - \alpha)p(a) + \alpha u(a).$$

Also write

$$h_\theta(a) = \nabla_\theta \log \pi_\theta(a | q), \quad R(a) = R(q, a).$$

(i) The old-policy target gradient has the form

$$g_{\text{old}} = \mathbb{E}_{a \sim p} [h_\theta(a) A_p(a)],$$

where A_p denotes the old-policy advantage. Since $m(a) > 0$ whenever $p(a) > 0$, we can rewrite this as

$$g_{\text{old}} = \mathbb{E}_{a \sim m} \left[\frac{p(a)}{m(a)} h_\theta(a) A_p(a) \right].$$

Therefore the importance weight is

$$w(a) = \frac{p(a)}{m(a)} = \frac{\pi_{\text{old}}(a | q)}{(1 - \alpha)\pi_{\text{old}}(a | q) + \alpha\pi_{\text{unif}}(a | q)}.$$

For samples $a_i \sim m$, the importance-corrected estimator is

$$\hat{g}_{\text{mix}} = \frac{1}{K} \sum_{i=1}^K w_i \nabla_\theta \log \pi_\theta(a_i | q) \hat{A}_i^{(p)},$$

where $w_i = w(a_i)$, and $\hat{A}_i^{(p)}$ should be an estimate of the old-policy advantage, not the mixture-policy advantage.

(ii) As $\alpha \rightarrow 0$,

$$m(a) \rightarrow p(a), \quad w(a) = \frac{p(a)}{m(a)} \rightarrow 1.$$

Therefore, for the same realised actions,

$$\hat{g}_{\text{mix}} \rightarrow \frac{1}{K} \sum_{i=1}^K \nabla_\theta \log \pi_\theta(a_i | q) \hat{A}_i,$$

which is the GRPO estimator.

(iii-a) By linearity of expectation,

$$V^{\pi_{\text{mix}}}(q) = (1 - \alpha)V^{\pi_{\text{old}}}(q) + \alpha V^{\pi_{\text{unif}}}(q).$$

(iii-b) The importance weights correct the outer sampling distribution, but they do not by themselves make the unweighted group mean $\bar{r}$ estimate $V^{\pi_{\text{old}}}(q)$. The unweighted mean estimates $V^{\pi_{\text{mix}}}(q)$. The population old-policy advantage is

$$A_p(a) = \frac{R(q, a) - V^{\pi_{\text{old}}}(q)}{\sigma_p}, \quad \sigma_p^2 = \text{Var}_{a \sim \pi_{\text{old}}(\cdot|q)}[R(q, a)].$$

Thus the corrected estimator should use

$$\hat{g}_{\text{mix}} = \frac{1}{K} \sum_{i=1}^K w_i \nabla_{\theta} \log \pi_{\theta}(a_i | q) \frac{r_i - V^{\pi_{\text{old}}}(q)}{\sigma_p}.$$

If $V^{\pi_{\text{old}}}$ and σ_p are unknown, the natural importance-weighted plug-in estimates are

$$\bar{r}_w = \frac{\sum_{j=1}^K w_j r_j}{\sum_{j=1}^K w_j}, \quad \sigma_w^2 = \frac{\sum_{j=1}^K w_j (r_j - \bar{r}_w)^2}{\sum_{j=1}^K w_j},$$

giving the reweighted advantage

$$\hat{A}_i^{(w)} = \frac{r_i - \bar{r}_w}{\sigma_w}.$$

Strict finite- K unbiasedness requires using the population $V^{\pi_{\text{old}}}, \sigma_p$, or estimating them from an independent or leave-one-out batch. The self-normalised plug-in version above is consistent but not exactly unbiased at finite K .

To see the baseline issue explicitly, if one instead centres by $V^{\pi_{\text{mix}}}$, then

$$\mathbb{E}_m[w h_{\theta}(a)(R(a) - V^{\pi_{\text{mix}}})] = \mathbb{E}_p[h_{\theta}(a)(R(a) - V^{\pi_{\text{old}}})] + (V^{\pi_{\text{old}}} - V^{\pi_{\text{mix}}}) \mathbb{E}_p[h_{\theta}(a)].$$

The second term is not generally zero unless the score is evaluated exactly under the same policy as the sampling distribution.

(iv) Ignoring the additional finite- K noise from estimating the baseline and standard deviation, define

$$Y(a) = h_{\theta}(a) A_p(a).$$

Then

$$\text{Var}(\hat{g}_{\text{mix}}) = \frac{1}{K} \text{Var}_{a \sim m}[w(a)Y(a)], \quad \text{Var}(\hat{g}_{\text{GRPO}}) = \frac{1}{K} \text{Var}_{a \sim p}[Y(a)].$$

Since

$$\mathbb{E}_m[wY] = \mathbb{E}_p[Y],$$

the difference in second moments along any parameter direction v is

$$\mathbb{E}_m[w^2(v^{\top} Y)^2] - \mathbb{E}_p[(v^{\top} Y)^2] = \mathbb{E}_p[(w - 1)(v^{\top} Y)^2].$$

Moreover,

$$w(a) - 1 = \frac{p(a) - m(a)}{m(a)} = \frac{\alpha(p(a) - u(a))}{m(a)}.$$

Thus mixture sampling increases the variance when the large squared advantages $(R(a) - V^{\pi_{\text{old}}})^2$ are concentrated in regions where $p(a) > u(a)$, because then $w(a) > 1$. It decreases the variance when the large squared advantages are concentrated in regions where $u(a) > p(a)$, because the uniform component oversamples those actions and the importance weight satisfies $w(a) < 1$.

For the raw, unweighted reward variance under the mixture,

$$\text{Var}_m(R) = (1 - \alpha) \text{Var}_p(R) + \alpha \text{Var}_u(R) + \alpha(1 - \alpha) (V^{\pi_{\text{old}}}(q) - V^{\pi_{\text{unif}}}(q))^2.$$

Therefore

$$\text{Var}_m(R) - \text{Var}_p(R) = \alpha \left[\text{Var}_u(R) - \text{Var}_p(R) + (1 - \alpha) (V^{\pi_{\text{old}}}(q) - V^{\pi_{\text{unif}}}(q))^2 \right].$$

So the mixture increases raw advantage variance if

$$\text{Var}_u(R) + (1 - \alpha) (V^{\pi_{\text{old}}}(q) - V^{\pi_{\text{unif}}}(q))^2 > \text{Var}_p(R),$$

for example if uniformly sampled responses have highly variable rewards. It decreases raw advantage variance if the reverse inequality holds, for example if the old policy samples both high- and low-reward responses but the uniform distribution mostly samples responses with rewards close to the old-policy mean.

(v-a) If each per-token ratio is $c \in (0, 1)$, then the full-response importance weight is

$$w_i = c^T = \exp(T \log c).$$

Since $\log c < 0$,

$$w_i \rightarrow 0 \quad \text{exponentially fast as } T \rightarrow \infty.$$

A ratio $c < 1$ occurs for tokens whose probability is increased by the uniform component, i.e. when

$$\pi_{\text{mix}}(a_{i,t} \mid q, a_{i,<t}) > \pi_{\text{old}}(a_{i,t} \mid q, a_{i,<t}).$$

This is typical for exploratory tokens that were unlikely under π_{old} , though it is not true for every token.

(v-b) Weight collapse makes the estimator unreliable because long responses receive exponentially small weights, causing their gradient contributions to vanish or underflow numerically. In the realistic case where token ratios vary across samples, the estimator is dominated by a small number of trajectories with non-negligible weights, giving very small effective sample size and high variance.

One concrete fix is to use truncated or self-normalised importance weights, for example

$$\tilde{w}_i = \frac{\min(w_i, M)}{\sum_{j=1}^K \min(w_j, M)}.$$

This reduces variance and avoids domination by extreme weights, but it introduces bias because $\tilde{w}_i$ no longer gives an exactly unbiased importance sampling estimator.

Another fix is to make the proposal closer to the old policy, for example by using a smaller α , or by enforcing a per-token bound

$$\left| \log \frac{\pi_{\text{old}}(a_t \mid q, a_{<t})}{\pi_{\text{mix}}(a_t \mid q, a_{<t})} \right| \leq \frac{\delta}{T}.$$

Then the full-response log weight is bounded by δ . This preserves more stable weights, but trades off exploration because π_{mix} is no longer allowed to move far from π_{old} .